

# Dokumen Teknis Backend: GENBA-HYOKA

Dokumen ini merangkum rancangan teknis untuk sistem pengawasan dan evaluasi operasional lapangan GENBA-HYOKA, menggunakan ekosistem Bun dan Elysia.js.

---

## BAB 1: Ringkasan Proyek

### 1.1 Latar Belakang dan Filosofi Sistem

Dalam manajemen operasional perusahaan, terdapat perbedaan cara kerja yang nyata antara kegiatan di dalam kantor dan di lapangan. Kegiatan kantor (seperti administrasi) biasanya lebih mudah dipantau. Sebaliknya, kegiatan lapangan—seperti inspeksi gedung, pengecekan kualitas barang, atau kunjungan tim sales—memiliki tantangan tersendiri karena dilakukan jauh dari jangkauan pengawasan langsung.

Jarak fisik ini seringkali menyebabkan kendala komunikasi. Laporan yang terlambat atau data yang tidak tercatat dengan rapi menyulitkan perusahaan dalam menilai kinerja pegawai secara adil. GENBA-HYOKA dibuat untuk mengatasi masalah tersebut. Nama ini diambil dari bahasa Jepang:

- **Genba (現場):** Lokasi kerja nyata, tempat masalah atau fakta ditemukan.
- **Hyoka (評価):** Penilaian, yaitu proses pemberian nilai berdasarkan fakta di lapangan tersebut.

### 1.2 Masalah yang Dihadapi dan Solusi

Saat ini, perusahaan yang memiliki banyak aktivitas di lapangan menghadapi tiga kendala utama:

1. **Sulit Memantau Kegiatan secara Langsung:** Pihak kantor tidak bisa melihat apa yang sedang terjadi di lapangan saat itu juga secara akurat.
2. **Informasi yang Tidak Terhubung:** Informasi seringkali tertahan di satu pihak saja. Misalnya, temuan masalah di lapangan tidak sampai ke manajer dengan cepat, sehingga arahan balik pun terlambat diberikan.
3. **Hasil Kegiatan yang Terpencar dan Sulit Dirangkum:** Hasil kerja lapangan seringkali tersimpan dalam format yang berbeda-beda dan tidak rapi, sehingga butuh waktu lama untuk mengumpulkannya menjadi laporan penilaian kinerja.

Sebagai solusi, GENBA-HYOKA menyediakan aplikasi web dan mobile yang memungkinkan pengawasan secara langsung. Manajer bisa memberikan masukan atau revisi saat itu juga, dan sistem akan otomatis merangkum semua aktivitas lapangan menjadi nilai kinerja yang objektif.

### 1.3 Ruang Lingkup Sistem (Cakupan Kerja)

Agar pengembangan tetap fokus pada kebutuhan utama, sistem ini dibatasi hanya pada empat kelompok fungsi:

- **Pengaturan Akses Pengguna:** Mengatur pendaftaran akun dan membatasi fitur apa saja yang boleh dibuka oleh pegawai sesuai jabatannya.
- **Pengelolaan Administrasi:** Tempat untuk mengatur profil perusahaan dan membuat format laporan digital yang bisa disesuaikan dengan kebutuhan (dinamis).
- **Kegiatan Operasional:** Bagian utama untuk pembagian tugas, pengisian laporan dari lapangan, hingga pelaporan jika ada masalah teknis di lokasi kerja.
- **Penilaian dan Evaluasi:** Proses bagi manajer untuk memeriksa laporan, memberikan masukan, melihat rangkuman nilai, serta mengunduh hasil kerja dalam format PDF atau Excel.

### 1.4 Standar Kualitas Sistem

Sistem ini dibangun dengan standar kualitas sebagai berikut:

- **Kemudahan Akses:** Sistem harus stabil dan bisa digunakan selama ada jaringan internet yang normal di lokasi kerja.
- **Kecepatan Respon:** Aplikasi harus bereaksi dengan cepat saat digunakan, agar pegawai tidak membuang waktu menunggu proses pemuatan (loading) yang lama.
- **Keamanan Data:** Semua data yang dikirim dipastikan aman melalui pengkodean rahasia (enkripsi). Setiap perubahan data penting akan dicatat riwayatnya (audit) agar bisa dilacak jika terjadi kesalahan.
- **Pencadangan Data:** Sistem akan menyimpan salinan data secara otomatis untuk mencegah kehilangan informasi jika terjadi gangguan pada server.

### 1.5 Asumsi dan Ketergantungan Infrastruktur

1. **Akses Jaringan:** Pengguna diasumsikan memiliki konektivitas internet yang memadai untuk mengirimkan data teks dan lampiran foto dari lokasi kerja lapangan.
2. **Kompatibilitas Perangkat:** Pengguna mengakses sistem melalui peramban (browser) modern yang mendukung teknologi komunikasi dua arah secara instan.
3. **Lingkungan Server:** Sistem bergantung pada server yang mendukung ekosistem Bun dan basis data PostgreSQL untuk menangani data yang terstruktur maupun yang fleksibel.

### 1.6 Penjelasan Istilah (Glosarium)

1. **API:** Cara aplikasi berkomunikasi dengan server pusat untuk bertukar informasi.
2. **Audit Log:** Catatan otomatis tentang siapa yang mengubah data, apa yang diubah, dan kapan hal itu dilakukan.
3. **JSONB:** Cara penyimpanan data di dalam sistem yang memungkinkan bentuk laporan diubah-ubah dengan mudah tanpa merusak program utama.

4. **Modular Monolith:** Cara menyusun program dengan membaginya menjadi bagian-bagian kecil agar mudah diperbaiki, namun tetap berjalan dalam satu sistem yang utuh.
5. **RBAC:** Aturan keamanan yang memastikan pengguna hanya bisa mengakses fitur yang sesuai dengan tugasnya.
6. **WebSocket:** Teknologi yang memungkinkan server mengirimkan pemberitahuan (notifikasi) secara instan ke perangkat pengguna tanpa harus diminta terlebih dahulu.

## 1.7 Profil Pengguna

Sistem GENBA-HYOKA membagi interaksi pengguna ke dalam tiga profil utama untuk menjamin efisiensi alur kerja operasional dan keamanan data:

- **Administrator Aplikasi:** Pengelola tingkat tinggi yang bertanggung jawab atas konfigurasi fundamental sistem. Fokus utamanya adalah pengelolaan hak akses (RBAC), penyusunan templat definisi tugas digital, serta pemeliharaan integritas profil perusahaan dan log audit sistem.
- **Manajer / Supervisor :** Otoritas operasional yang menjembatani instruksi strategis dengan eksekusi lapangan. Peran ini bertanggung jawab melakukan pendelegasian tugas kepada pegawai, meninjau validitas hasil pekerjaan yang masuk, memberikan instruksi perbaikan (revisi), serta memantau dan menilai hasil pekerjaan yang dilakukan.
- **Pegawai Lapangan :** Pelaksana teknis yang berinteraksi langsung dengan fakta di lokasi kerja. Bertanggung jawab mengumpulkan data lapangan secara akurat melalui formulir digital, melaporkan kendala operasional secara *real-time*, dan melakukan pembaruan laporan berdasarkan masukan dari supervisor.

---

## BAB 2: Arsitektur dan Desain Sistem

### 2.1 Struktur Proyek dan Pola Arsitektur (Modular Monolith)

Sistem GENBA-HYOKA diimplementasikan menggunakan pola **Modular Monolith**. Pendekatan ini mengorganisir kode program ke dalam modul-modul mandiri berdasarkan fungsi bisnis, namun tetap berada dalam satu unit distribusi (satu proses runtime Bun) untuk efisiensi operasional.

Struktur direktori utama yang digunakan:

genba-hyoka-backend/

```
|— src/           # Direktori utama kode sumber
|
| |— modules/     # Implementasi logika bisnis per modul
```

```

| | └─ auth/          # Modul keamanan dan autentikasi
| | └─ users-manajemnt/      # Modul manajemen pengguna dan hak akses
| | └─ tasks-manajemnt/      # Modul pengelolaan siklus hidup tugas
| | └─ reports-manajemnt/     # Modul pengolahan laporan dan ekspor data
| | └─ issues-manajemnt/      # Modul pelaporan kendala (Problem Report)
| └─ db/                  # Konfigurasi basis data dan skema Drizzle ORM
| └─ middlewares/          # Fungsi validasi (JWT, RBAC) dan pengamanan
| └─ libs/                  # Pustaka pembantu (Abstraksi storage, dll)
| └─ utils/                 # Pustaka fungsi global yang dapat digunakan berulang
| └─ index.ts              # Entry point aplikasi (Elysia.js)
└─ uploads/                # Penyimpanan fisik berkas (Foto dan file laporan)
└─ tests/                  # Skrip pengujian otomatis (Unit & Integration)
└─ .env                    # Variabel lingkungan dan kredensial sistem
└─ package.json            # Manifest dependensi proyek

```

## 2.2 Manajemen Berkas dan Abstraksi Penyimpanan

Sistem memisahkan logika pengolahan berkas dengan lokasi penyimpanan fisik:

1. **Logika Pengolahan (`src/libs/storage.ts`):** Menangani standarisasi berkas (pembatasan ukuran, validasi tipe file). Dibuat abstrak agar sistem dapat berpindah dari penyimpanan lokal ke Cloud Storage tanpa mengubah logika bisnis.
2. **Penyimpanan Fisik (`uploads/`):** Direktori khusus untuk menyimpan data statis hasil unggahan pengguna (seperti foto lapangan), terpisah dari kode sumber (`src`) guna meningkatkan keamanan dan mempermudah pencadangan.

## 2.3 Pemodelan Data Relasional dan Dinamis

Implementasi basis data pada PostgreSQL menangani dua karakteristik:

- **Data Terstruktur:** Menggunakan relasi antar tabel dengan identifikasi **UUID** (Universally Unique Identifier) untuk keamanan identitas.
- **Data Dinamis (JSONB):** Pemanfaatan tipe **JSONB** (seperti atribut `form_schema` dan `form_data`) memungkinkan sistem menyimpan formulir laporan fleksibel yang dapat disesuaikan tanpa mengubah skema tabel inti.

## 2.4 Mekanisme Keamanan dan Otorisasi

Keamanan data pada sistem GENBA-HYOKA dirancang dengan prinsip *Zero Trust* yang dipadukan dengan optimalisasi performa tinggi ( *high-performance execution*). Untuk menghindari *bottleneck* pada database saat aplikasi diakses oleh ratusan pegawai lapangan secara bersamaan, sistem perlindungan data diimplementasikan melalui tiga pilar utama:

- **Autentikasi Berbasis Token (JWT):** Identitas pengguna divalidasi menggunakan tiket digital berbatas waktu (*stateless validation*).
- **Middleware Verifikasi (Bitmasking RBAC):** Menjalankan kontrol akses berbasis peran ( *Role-Based Access Control*) langsung di level memori prosesor tanpa melakukan kueri ke database.
- **Enkripsi Kredensial:** Menggunakan algoritma *hashing* standar industri untuk melindungi informasi sensitif pengguna.

### 2.4.1 Kebijakan Dual-Token Strategy

Untuk menyeimbangkan antara keamanan data yang ketat dan kenyamanan pengguna di lapangan (*User Experience*), sistem mengimplementasikan strategi dua lapis token dengan ketentuan durasi sebagai berikut:

- **Access Token (Short-Lived - 1 Jam):** Digunakan sebagai kredensial utama dalam setiap permintaan API. Durasi 1 jam dipilih secara pragmatis untuk mengakomodasi pola penggunaan aplikasi operasional yang bersifat periodik (tidak diakses terus-menerus), sehingga meminimalisir beban jaringan dan menghemat konsumsi daya perangkat lapangan.
- **Refresh Token (Long-Lived - 9 Jam):** Digunakan secara khusus untuk memperbarui *Access Token* tanpa melalui proses *login* manual. Durasi 9 jam disesuaikan dengan rata-rata siklus waktu kerja harian di Indonesia. Kebijakan ini memastikan bahwa akses pengguna akan secara otomatis kedaluwarsa setelah jam kerja berakhir, sehingga memitigasi risiko penyalahgunaan akun di luar waktu operasional.

### 2.4.2 Mekanisme Penyimpanan dan Rotasi Sesi

Untuk mendukung alur *Refresh Token* di atas, sistem mengadopsi skema *Session Update* pada basis data guna menjamin efisiensi penyimpanan dan integritas sesi pengguna:

- **Pembaruan Berkala (Update Logic):** Saat terjadi proses *Silent Refresh*, sistem tidak akan menambah baris baru. Sistem hanya akan melakukan pembaruan (*update*) pada nilai token dan `expired_at` pada baris sesi yang sudah ada.
- **Terminasi Otomatis:** Apabila durasi *Refresh Token* (9 jam) telah terlampaui, baris sesi

terkait akan dianggap tidak valid. Hal ini akan memicu aplikasi *client* untuk memaksa (*force-logout*) dan mengarahkan pengguna kembali ke halaman *login* utama.

### 2.4.3 Mekanisme Validasi Akses Api Dengan Bitmasking

GENBA-HYOKA mendukung struktur organisasi yang dinamis, di mana satu pegawai dapat memegang lebih dari satu jabatan fungsional sekaligus (*Multi-Role*). Peran (*Role*) ini berfungsi sebagai parameter sistem untuk mendistribusikan tugas operasional, sekaligus mengunci hak akses pengguna terhadap manipulasi data. Hal ini memastikan bahwa pemanggilan API hanya dapat dilakukan oleh pengguna yang memiliki otoritas yang tepat.

Alih-alih melakukan kueri basis data yang berat pada setiap permintaan API, sistem memvalidasi akses dengan mencocokkan peran pengguna terhadap sebuah konstanta (*constant*) yang telah didefinisikan langsung di dalam kode *Backend*. Konstanta ini menggunakan nilai desimal (Bitmask) untuk merepresentasikan hak akses:

| Method | Bit Value | Keterangan                 |
|--------|-----------|----------------------------|
| READ   | 1         | Akses melihat/membaca data |
| CREATE | 2         | Akses membuat data baru    |
| UPDATE | 4         | Akses mengubah data        |
| DELETE | 8         | Akses menghapus data       |

#### Proses Kerja:

1. **Saat Login:** Sistem mengidentifikasi tipe *Role* yang dimiliki pengguna (misal: manager), kemudian mengambil hak akses dari konstanta yang sesuai, dan menyimpannya ke dalam token JWT (*Payload*).
2. **Saat Pemanggilan API:** Pengguna mengirimkan token JWT pada *Header* permintaan.
3. **Validasi Middleware:** Sistem membaca token JWT dan menggunakan operasi *Bitwise AND* (&) untuk memastikan apakah hak akses yang tertanam di token tersebut memenuhi syarat minimal operasi API yang sedang dipanggil. Jika sesuai, proses dilanjutkan; jika tidak, akses ditolak (403 Forbidden).

### 2.4.4 Manajemen Kunci Rahasia (Secret Key Management)

Keabsahan setiap token JWT dalam sistem GENBA-HYOKA sangat bergantung pada kerahasiaan *Secret Key*. Kunci ini bertindak sebagai satu-satunya instrumen untuk memverifikasi tanda tangan digital (*Signature*) pada setiap permintaan API.

#### Kebijakan Pengelolaan Kunci:

1. **Penyimpanan Terisolasi:** *Secret Key* dilarang keras untuk ditulis langsung di dalam kode program (*hardcoded*). Kunci wajib disimpan dalam variabel lingkungan (*Environment Variables*) melalui file `.env` di sisi *Backend*.
2. **Eksklusivitas Server:** *Secret Key* hanya dikelola dan diketahui oleh server *Backend*. Sisi *Frontend* (React/Vite) tidak diizinkan memiliki akses atau menyimpan kunci ini dalam bentuk apa pun.
3. **Keamanan Repositori:** File `.env` wajib didaftarkan ke dalam file `.gitignore` untuk mencegah kebocoran kunci ke dalam repositori kode (Git/GitHub)

# Konfigurasi Keamanan JWT (Server Side Only)

```
JWT_SECRET=8f3a9b2c...[string_acak_panjang_minimal_32_karakter]
JWT_EXPIRES_IN=1h
JWT_REFRESH_EXPIRES_IN=9h
JWT_REFRESH_SECRET=7d2e1f4...[kunci_berbeda_untuk_refresh_token]
```

**Mekanisme Verifikasi:** Setiap kali permintaan masuk, sistem akan mengambil `JWT_SECRET` dari memori server untuk melakukan kalkulasi ulang terhadap *Header* dan *Payload* yang dikirim klien. Jika hasil kalkulasi server tidak identik dengan *Signature* pada token, maka permintaan akan segera ditolak (*401 Unauthorized*) tanpa diproses lebih lanjut.

## 2.5 Arsitektur Komunikasi Real-time (WebSocket)

Sistem menggunakan protokol WebSocket (WSS) untuk memfasilitasi pengiriman notifikasi dan pembaruan data secara instan. Untuk menjamin keamanan dan performa, arsitektur ini didukung oleh mekanisme validasi berbasis tabel pada PostgreSQL.

### 2.5.1 Strategi Topic-Based Pub/Sub

Untuk efisiensi distribusi pesan, sistem menggunakan dua jenis saluran (*channel*) komunikasi:

- **User Channel (user : {user\_id}):** Mengirim notifikasi privat seperti penugasan individu.
- **Role Channel (role : {role\_id}):** Mengirim notifikasi kelompok, seperti laporan kendala (*problem report*) kepada seluruh pengguna dengan peran "Manager" atau "Supervisor".

### 2.5.2 Mekanisme Ketangguhan (Resilience)

Sistem dirancang untuk tetap stabil pada kondisi jaringan lapangan yang tidak menentu melalui:

- **Heartbeat (Ping-Pong):** Deteksi koneksi aktif setiap 30 detik untuk mencegah penumpukan koneksi pasif (*zombie connection*) pada memori server.

- **Exponential Backoff:** Strategi percobaan ulang koneksi dengan jeda waktu yang meningkat secara bertahap guna menghindari beban berlebih pada server.
- **Missed Message Sync:** Jika koneksi terputus, aplikasi akan melakukan sinkronisasi otomatis ke tabel `notifications` melalui API HTTP segera setelah terhubung kembali.

### 2.5.3 Keamanan Handshake (Tabel `ws_tickets`)

Mengingat keterbatasan protokol WebSocket dalam mengirimkan *header* kustom pada beberapa lingkungan *client-side*, sistem menggunakan tabel `ws_tickets` sebagai lapisan keamanan sebelum pipa komunikasi dibuka:

| Nama Atribut            | Tipe Data | Panjang | Index          | Deskripsi Fungsi                                                      |
|-------------------------|-----------|---------|----------------|-----------------------------------------------------------------------|
| <code>id</code>         | UUID      | -       | <b>PK</b>      | Tiket unik yang dikirim ke klien untuk proses <i>handshake</i> .      |
| <code>user_id</code>    | UUID      | -       | -              | Identitas user yang akan membuka koneksi (FK ke <code>users</code> ). |
| <code>expires_at</code> | Timestamp | -       | <b>B-Tree-</b> | Batas waktu berlaku tiket (direkomendasikan 60 detik).                |

- **Fungsi Tabel:** Menyimpan token sementara (One-Time Ticket/OTT) yang menghubungkan identitas pengguna dari jalur REST API ke jalur WebSocket.
- **Mekanisme Kerja:**
  1. Klien melakukan permintaan tiket melalui API HTTP yang terautentikasi JWT.
  2. Server men- *generate* UUID unik, menyimpannya ke tabel `ws_tickets` bersama ID pengguna, dan menetapkan waktu kedaluwarsa (60 detik).
  3. Saat jabat tangan (*handshake*) WebSocket dimulai, server memvalidasi UUID tersebut pada tabel.
  4. Jika valid, server melakukan "Upgrade" protokol menjadi WebSocket dan **segera menghapus** data tiket dari tabel untuk mencegah penggunaan ulang.

## 2.6 Standar Atribut Audit dan Mekanisme Soft Delete

Setiap entitas dalam basis data memiliki 6 atribut audit standar untuk menjamin integritas riwayat dan keamanan data:

1. `created_at`: Waktu presisi pembuatan data.
2. `created_by`: Identitas pengguna pembuat (UUID).
3. `updated_at`: Waktu terakhir kali data diubah.
4. `updated_by`: Identitas pengguna yang melakukan perubahan terakhir.
5. `deleted_at`: Waktu saat data dinonaktifkan secara logis.
6. `deleted_by`: Identitas pengguna yang menghapus data.

*Data tidak akan dihapus secara permanen jika data tersebut memiliki relasi dengan data lain maka akan dilakukan flagging data (soft delete) sebagai tanda data sudah dihapus, jika ternyata tidak ada relasi maka data dapat dihapus secara permanen (hard delete). Seluruh query standar hanya memanggil data dengan `deleted_at IS NULL`.*

## 2.7 Desain Basis Data (ERD) dan Logika Bisnis

### 2.7.1 Visualisasi Relasi Entitas (ERD)

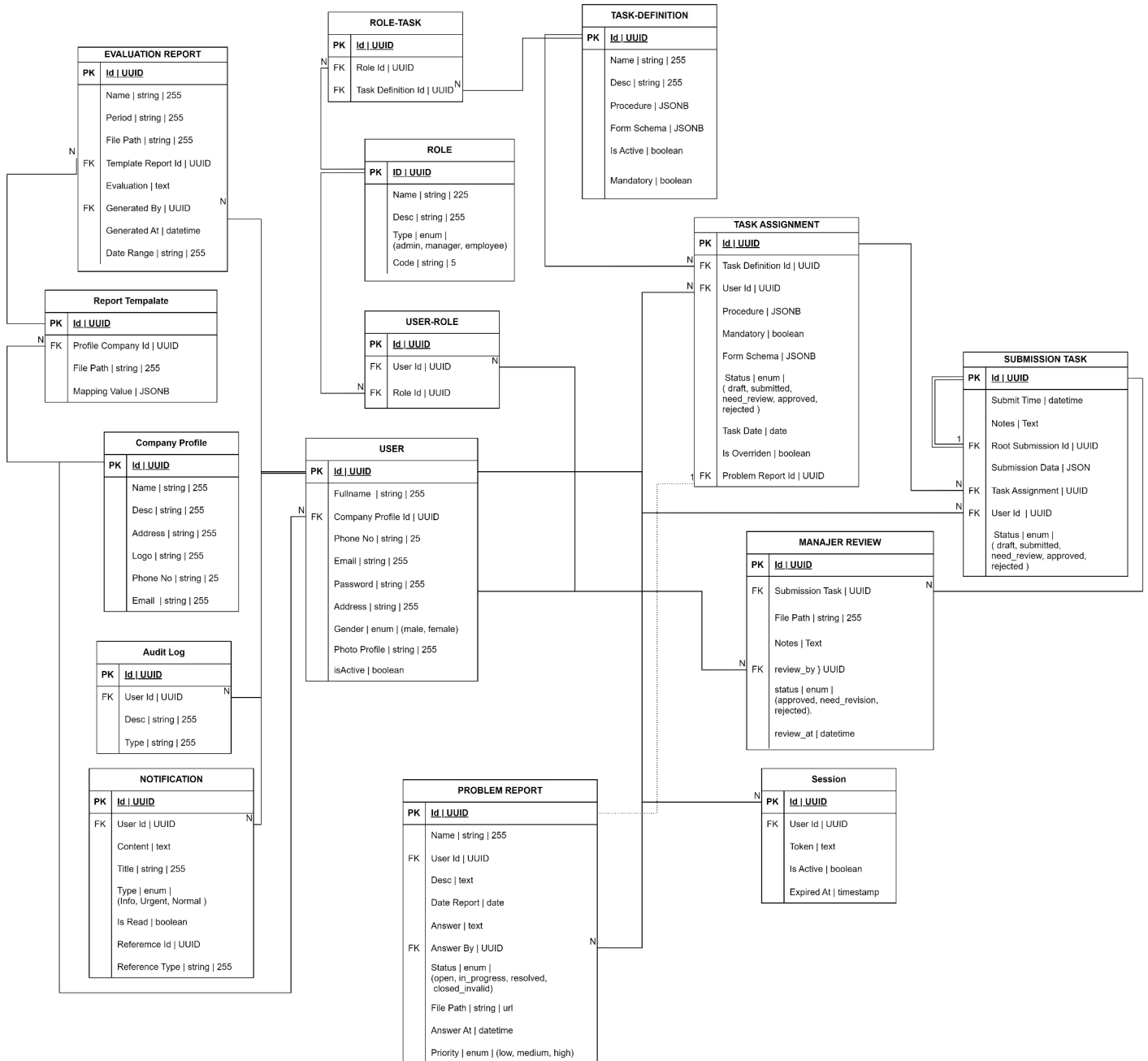


Figure 1: ERD Genba Hyoka

### 2.7.2 Logika Klaster Bisnis (Movement Logic & Workflow)

Sistem Genba-Hyoka tidak bersifat statis. Basis data dirancang untuk bergerak dan bereaksi terhadap interaksi pengguna. Berikut adalah kronologi pergerakan data pada masing-masing klaster operasional:

**Klaster 1: Manajemen Akses & Keamanan (The Gatekeeper Flow)** Klaster ini menangani gerbang utama masuknya pengguna ke dalam sistem. Fokus utamanya adalah mengubah data identitas dari basis data menjadi klaim otorisasi yang ringkas dan siap pakai di dalam memori.

#### Langkah-langkah Operasional:

1. **Validasi Kredensial (Identity Check):** Sistem menerima email dan kata sandi dari klien. Pencarian identitas dilakukan pada tabel `users`. Kata sandi kemudian divalidasi menggunakan algoritma *hashing* internal milik Bun untuk menjamin keamanan.
2. **Kalkulasi Hak Akses Efektif (Flattening Process):** Jika identitas valid, sistem melakukan kueri *JOIN* ringan ke tabel `user_roles` dan `roles` untuk mengidentifikasi tipe peran pengguna (misal: `admin`, `manager`, `employee`). Sistem kemudian mencocokkannya dengan aturan hak akses yang telah didefinisikan dalam bentuk konstanta di *Backend*. Jika pengguna memiliki lebih dari satu peran (*Multi-Role*), sistem melakukan operasi *Bitwise OR* (`|`) untuk melebur seluruh hak akses tersebut menjadi satu objek "Hak Akses Efektif".
3. **Penerbitan Token (JWT Signing):** Hasil kalkulasi hak akses (objek `prn`) disematkan ke dalam *payload* JWT bersama dengan informasi sesi pengguna. Sistem kemudian menerbitkan dua jenis token: *Access Token* yang ditandatangani menggunakan `JWT_SECRET`, dan *Refresh Token* yang ditandatangani menggunakan `JWT_REFRESH_SECRET` (tersimpan aman di `.env`).
4. **Validasi Middleware (Stateless Verification):** Pada setiap permintaan API berikutnya, *Middleware* tidak perlu lagi melakukan kueri hak akses ke basis data. *Middleware* murni berjalan secara *stateless* dan cukup melakukan:
  - **Verifikasi Signature:** Memastikan token asli, tidak diubah oleh pihak ketiga, dan belum kedaluwarsa.
  - **Bitwise Check:** Melakukan operasi *Bitwise AND* (`&`) langsung di level memori antara nilai `prn` di dalam token dengan syarat akses *endpoint* yang sedang dituju.

#### Klaster 2: Definisi dan Penugasan Tugas (The Delegation Flow)

Klaster ini mengubah templat atau aturan kosong menjadi instruksi kerja nyata bagi pegawai.

1. **Pembuatan Cetak Biru:** Admin merancang struktur laporan menggunakan format JSONB di `task_definitions.form_schema`, menyusun aturan pengisiannya di `task_definitions.procedures` dan menautkannya ke peran tertentu melalui `role_tasks`.
2. **Pendelegasian Tugas:** Manajer memilih pegawai dan membuat baris baru di tabel `task_assignments`, lengkap dengan `form_schema` dan `procedures`.

3. **Distribusi Notifikasi:** Begitu penugasan disimpan, sistem secara otomatis (*trigger*) membuat baris di tabel `notifications` dengan tipe `Normal` untuk memberi tahu perangkat *mobile* pegawai bahwa ada tugas baru yang menanti.

#### Klaster 3: Pelaksanaan dan Pelaporan (The Field Execution Flow)

Klaster ini merekam jejak kerja pegawai dengan prinsip *Immutability* (data tidak boleh ditimpa untuk menjaga integritas audit).

1. **Penyimpanan Draf:** Saat pegawai mulai mengisi formulir di lapangan, data disimpan dengan status `draft` pada tabel `submission_tasks`. Data ini belum masuk ke radar manajer.
2. **Pengiriman Laporan (Submission):** Saat dikirim, status berubah menjadi `submitted` dengan `version_number: 1`. Laporan ini sekarang terkunci (*read-only*) bagi pegawai.
3. **Pelaporan Kendala (Opsional):** Jika terjadi hambatan fisik (misal: alat rusak), pegawai tidak mengosongkan tugas. Mereka membuat baris di `problem_reports` (status `open`), yang langsung memicu notifikasi `Urgent` ke manajer.
4. **Siklus Versi (Versioning Loop):** Jika laporan dikembalikan oleh manajer (revisi), pegawai membuat perbaikan yang akan disimpan sebagai **baris baru** (`version_number: 2`). Baris baru ini direlasikan dengan atribut `root_submission_id` pada data utama yang di perbaiki.

#### Klaster 4: Evaluasi dan Penilaian (The Resolution Flow)

Klaster ini adalah titik temu antara hasil lapangan dan pengawasan manajerial yang berujung pada nilai performa.

1. **Pencatatan Ulasan (Review Entry):** Manajer memeriksa laporan yang masuk. Keputusan manajer tidak disimpan di tabel laporan, melainkan membentuk baris baru di `manager_reviews` beserta catatan detailnya.
2. **Gerbang Sinkronisasi Status (State Sync):** *Backend* bereaksi terhadap tabel ulasan tadi:
  - Jika `manager_reviews.status = need_revision`, maka `submission_tasks.status` otomatis berubah menjadi `need_reviewd` (membuka akses bagi pegawai untuk membuat Versi 2).
  - Jika `manager_reviews.status = approved`, maka `submission_tasks.status` berubah menjadi `approved`.
3. **Kemampuan Membuat Evaluasi:** Pada akhir periode, sistem dapat menampilkan hasil kinerja dari pegawai lapangan dan memberikan penilaian dalam bentuk text, dan membuat nya menjadi sebuah data laporan berdasarkan `report_templates` kemudian menyimpan nya dalam tabel `evaluation_reports`.

#### 2.7.3 Kamus Data (Data Dictionary)

**Catatan Arsitektur:** > Untuk menjamin integritas dan jejak audit (*audit trail*), setiap entitas/tabel dalam sistem secara otomatis mewarisi 6 atribut standar berikut:

- **`created_at / created_by`:** Mencatat waktu dan aktor pembuat data.

- **updated\_at / updated\_by:** Mencatat waktu dan aktor perubahan terakhir.
- **deleted\_at / deleted\_by:** Mendukung mekanisme Soft-Delete untuk keamanan data.

#### Ketentuan Indeks Gabungan (Composite Index)

Sistem menerapkan strategi indeks gabungan (Composite B-Tree) untuk mengoptimalkan performa kueri pada modul-modul operasional yang bersifat dinamis.

- **Atribut Tanda Composite 1\*:** Menandakan kolom tersebut digabungkan ke dalam satu unit indeks untuk mempercepat validasi otorisasi ganda dan penyaringan data di level Middleware.
- **Default Indexing Policy:** Untuk setiap tabel yang menyimpan data transaksi/kegiatan, sistem secara otomatis menerapkan indeks gabungan pada kolom **user\_id** dan **created\_at**.

---

Nama Tabel: **roles**

| Nama Atribut | Tipe Data       | Panjang                                 | Index | Keterangan                                                                  |
|--------------|-----------------|-----------------------------------------|-------|-----------------------------------------------------------------------------|
| id           | UUID            | -                                       | PK    | Identitas unik untuk setiap peran dalam sistem.                             |
| name         | String   unique | 255                                     |       | Nama dari peran tersebut (misalnya: Admin, Manajer, User).                  |
| description  | String          | 255                                     |       | Penjelasan singkat mengenai fungsi atau tanggung jawab dari peran tersebut. |
| type         | Enum            | (admin, manager, employee, super admin) |       | Tipe peran yang mewakili tugas dari peran tersebut.                         |
| code         | String          | 5                                       |       | Kode yang mewakili data peran                                               |

---

Nama Tabel: **user\_roles**

| Nama Atribut | Tipe Data | Panjang | Index               | Keterangan                                        |
|--------------|-----------|---------|---------------------|---------------------------------------------------|
| id           | UUID      | -       | <b>PK</b>           | Identitas unik untuk relasi antara user dan role. |
| user_id      | UUID      | -       | <b>Composite 1*</b> | Foreign Key (FK). Referensi ke tabel users.       |
| role_id      | UUID      | -       | <b>Composite 1*</b> | Foreign Key (FK). Referensi ke tabel roles.       |

---

Nama Tabel: **users**

| Nama Atribut       | Tipe Data       | Panjang | Index         | Keterangan                                            |
|--------------------|-----------------|---------|---------------|-------------------------------------------------------|
| id                 | UUID            | -       | <b>PK</b>     | Identitas unik pengguna.                              |
| full_name          | String   unique | 255     | <b>B-Tree</b> | Nama lengkap pengguna.                                |
| company_profile_id | UUID   unique   | -       | <b>B-Tree</b> | Foreign Key (FK). Relasi ke tabel company_profiles.   |
| phone_no           | String   unique | 25      |               | Nomor telepon/handphone pengguna.                     |
| email              | String   unique | 255     |               | Alamat email pengguna (sering digunakan untuk login). |
| password           | String          | 255     |               | Kata sandi akun pengguna (terenkripsi).               |

| Nama Atribut  | Tipe Data | Panjang          | Index | Keterangan                           |
|---------------|-----------|------------------|-------|--------------------------------------|
| address       | String    | 255              |       | Alamat tempat tinggal pengguna.      |
| gender        | Enum      | ( male, female ) |       | Informasi jenis kelamin              |
| photo_profile | String    | 255              |       | Lokasi penyimpanan file foto profil. |
| is_active     | Boolean   | -                |       | Status aktif pengguna (true/false).  |

Nama Tabel : **evaluation\_reports**

| Nama Atribut       | Tipe Data       | Panjang               | Index         | Deskripsi Fungsi                                                                                               |
|--------------------|-----------------|-----------------------|---------------|----------------------------------------------------------------------------------------------------------------|
| id                 | UUID            |                       | <b>PK</b>     | Identitas acak dan unik untuk setiap dokumen evaluasi.                                                         |
| name               | String   unique | 255                   | <b>B-Tree</b> | Nama atau judul dokumen laporan evaluasi.                                                                      |
| date_range         | String          | 255                   |               | Informasi rentang tanggal dari periode yang dipilih.                                                           |
| period             | Enum            | (hari, minggu, bulan) |               | Rentang waktu atau periode pelaksanaan evaluasi.                                                               |
| file_path          | String          | 255                   |               | Jalur penyimpanan (file path) atau URL yang merujuk ke lokasi fisik dokumen evaluasi di dalam folder uploads/. |
| report_template_id | UUID            |                       |               | Kunci tamu yang merujuk ke tabel                                                                               |

| Nama Atribut | Tipe Data | Panjang | Index         | Deskripsi Fungsi                                                                          |
|--------------|-----------|---------|---------------|-------------------------------------------------------------------------------------------|
|              |           |         |               | report_templates.<br>Berfungsi untuk mengidentifikasi format standar mana yang digunakan. |
| evaluation   | Text      |         |               | Data hasil evaluasi user.                                                                 |
| generated_by | UUID      |         | <b>B-Tree</b> | Id user yang melakukan pembuatan laporan evaluasi                                         |
| generated_at | DATETIME  |         |               | Waktu kapan dokumen dibuat.                                                               |

Nama Tabel: **report\_templates**

| Nama Atribut       | Tipe Data | Panjang | Index         | Keterangan                                                                     |
|--------------------|-----------|---------|---------------|--------------------------------------------------------------------------------|
| id                 | UUID      | -       | <b>PK</b>     | Primary Key (PK).<br>Identitas unik untuk setiap template report.              |
| company_profile_id | UUID      | -       | <b>B-Tree</b> | Foreign Key (FK).<br>Referensi ke tabel profil perusahaan terkait.             |
| file_path          | String    | 255     |               | Nama file path lokasi penyimpanan file template.                               |
| mapping_value      | JSONB     | -       |               | Data pemetaan nilai dalam format JSON biner untuk fleksibilitas struktur data. |

---

Nama Tabel: **company\_profiles**

| Nama Atribut | Tipe Data       | Panjang | Index  | Keterangan                                           |
|--------------|-----------------|---------|--------|------------------------------------------------------|
| id           | UUID            | -       | PK     | Identitas unik perusahaan.                           |
| name         | String   unique | 255     | B-Tree | Nama lengkap instansi atau perusahaan.               |
| description  | String          | 255     |        | Deskripsi singkat mengenai profil perusahaan.        |
| address      | String          | 255     |        | Alamat fisik kantor pusat perusahaan.                |
| logo         | String          | 255     |        | Nama file path lokasi penyimpanan logo perusahaan.   |
| phone_no     | String          | 25      |        | Nomor telepon resmi perusahaan yang dapat dihubungi. |
| email        | String          | 255     |        | Alamat email resmi korespondensi perusahaan.         |

---

Nama Tabel: **audit\_logs**

\*hanya ada created\_at dan created\_by

| Nama Atribut | Tipe Data | Panjang | Index        | Keterangan                                          |
|--------------|-----------|---------|--------------|-----------------------------------------------------|
| id           | UUID      | -       | PK           | Identitas unik untuk setiap baris log audit.        |
| user_id      | UUID      | -       | Composite 1* | Foreign Key (FK). Referensi ke users yang melakukan |

| Nama Atribut | Tipe Data | Panjang | Index               | Keterangan                                                        |
|--------------|-----------|---------|---------------------|-------------------------------------------------------------------|
|              |           |         |                     | aktivitas tersebut.                                               |
| description  | String    | 255     |                     | Deskripsi atau detail mengenai aktivitas yang dilakukan.          |
| type         | String    | 255     | <b>Composite 1*</b> | Jenis atau kategori aktivitas (misalnya: CREATE, UPDATE, DELETE). |

Nama Tabel: **notifications**

| Nama Atribut   | Tipe Data | Panjang                 | Index         | Keterangan                                                     |
|----------------|-----------|-------------------------|---------------|----------------------------------------------------------------|
| id             | UUID      | -                       | <b>PK</b>     | Identitas unik untuk setiap data notifikasi.                   |
| user_id        | UUID      | -                       |               | Foreign Key (FK). Referensi ke users yang menerima notifikasi. |
| content        | Text      | -                       |               | Isi pesan atau detail informasi dari notifikasi.               |
| title          | String    | 255                     |               | Judul atau subjek singkat dari notifikasi.                     |
| type           | Enum      | (info, urgent, normal). |               | Kategori urgensi notifikasi:                                   |
| is_read        | Boolean   |                         |               | Flag pesan sudah dibaca atau belum.                            |
| reference_id   | UUID      |                         |               | ID referensi ke data apa pemberitahuan ini berasal.            |
| reference_type | String    |                         | <b>B-Tree</b> | Nama table referensi                                           |

| Nama Atribut | Tipe Data | Panjang | Index | Keterangan                                      |
|--------------|-----------|---------|-------|-------------------------------------------------|
|              |           |         |       | agar mudah di tracing sumber data referensinya. |

---

Nama Tabel: **role\_tasks**

| Nama Atribut       | Tipe Data | Panjang | Index     | Keterangan                                             |
|--------------------|-----------|---------|-----------|--------------------------------------------------------|
| id                 | UUID      | -       | <b>PK</b> | Identitas unik untuk relasi antara role dan task.      |
| role_id            | UUID      | -       |           | Foreign Key (FK). Referensi ke tabel roles.            |
| task_definition_id | UUID      | -       |           | Foreign Key (FK). Referensi ke tabel task_definitions. |

---

Nama Tabel: **task\_assignments**

| Nama Atribut | Tipe Data | Panjang | Index     | Keterangan                                                                 |
|--------------|-----------|---------|-----------|----------------------------------------------------------------------------|
| id           | UUID      | -       | <b>PK</b> | Identitas unik untuk setiap assignment.                                    |
| mandatory    | boolean   | -       |           | Flag attribute yang digunakan apakah tugas ini harus dilakukan atau tidak. |
| form_schema  | JSONB     |         |           | Attribute json yang berisikan struktur dan tata cara pengisian tugas.      |
| procedures   | JSONB     | -       |           | Daftar langkah-                                                            |

| Nama Atribut       | Tipe Data | Panjang                                    | Index               | Keterangan                                                                                                                     |
|--------------------|-----------|--------------------------------------------|---------------------|--------------------------------------------------------------------------------------------------------------------------------|
|                    |           |                                            |                     | langkah atau prosedur kerja dalam format JSON biner.                                                                           |
| Status             | Enum      | (draft, submitted, need_reviewd, approved, |                     | Attribute yang berisikan status tugas, agar lebih mudah di query.                                                              |
| problem_report_id  | UUID      |                                            |                     | Isian optional ketika ada perubahan data maka agar mudah dilacak karena kendala apa maka id laporan kendala tersebut disimpan. |
| is_overridden      | boolean   |                                            |                     | Flag menandakan tugas ini ada perubahan atau tidak karena laporan kendala.                                                     |
| user_id            | UUID      | -                                          | <b>Composite 1*</b> | Foreign Key (FK). Referensi ke tabel users.                                                                                    |
| task_definition_id | UUID      | -                                          | <b>Composite 1*</b> | Foreign Key (FK). Referensi ke tabel task_definitions.                                                                         |
| task_date          | Datetime  |                                            |                     | Informasi tanggal tugas kerjakan.                                                                                              |

Nama Tabel: **task\_definitions**

| Nama Atribut | Tipe Data | Panjang | Index     | Keterangan           |
|--------------|-----------|---------|-----------|----------------------|
| id           | UUID      | -       | <b>PK</b> | Identitas unik untuk |

| Nama Atribut | Tipe Data | Panjang | Index         | Keterangan                                                          |
|--------------|-----------|---------|---------------|---------------------------------------------------------------------|
|              |           |         |               | setiap definisi tugas.                                              |
| name         | String    | 255     | <b>B-Tree</b> | Nama atau judul dari tugas yang didefinisikan.                      |
| description  | String    | 255     |               | Penjelasan atau deskripsi mengenai tugas tersebut.                  |
| procedures   | JSONB     | -       |               | Daftar langkah-langkah atau prosedur kerja dalam format JSON biner. |
| form_schema  | JSONB     | -       |               | Skema struktur formulir digital yang terkait dengan tugas tersebut. |
| is_active    | Boolean   | -       |               | Status keaktifan tugas (true = aktif, false = non-aktif).           |
| is_mandatory | Boolean   | -       |               | Menentukan apakah tugas ini wajib dikerjakan atau tidak.            |

---

Nama Tabel: **submission\_tasks**

| Nama Atribut       | Tipe Data | Panjang | Index     | Keterangan                                                    |
|--------------------|-----------|---------|-----------|---------------------------------------------------------------|
| id                 | UUID      | -       | <b>PK</b> | Identitas unik untuk setiap pengumpulan tugas.                |
| task_assignment_id | UUID      | -       |           | Foreign Key (FK). Referensi ke task_assignments yang terkait. |

| Nama Atribut       | Tipe Data | Panjang                                              | Index               | Keterangan                                                                                             |
|--------------------|-----------|------------------------------------------------------|---------------------|--------------------------------------------------------------------------------------------------------|
| submission_data    | JSONB     | -                                                    |                     | Data konten tugas yang dikumpulkan dalam format JSON.                                                  |
| version_number     | Integer   | -                                                    |                     | Urutan pengiriman (1, 2, 3, dst).                                                                      |
| status             | Enum      | (draft, submitted, need_review, approved, rejected). | <b>Composite 1*</b> | Status pengumpulan:                                                                                    |
| root_submission_id | UUID      | -                                                    |                     | FK (Nullable). Rantai penghubung ke id dari tugas pertamanya.                                          |
| notes              | Text      |                                                      |                     | Catatan pengerjaan tugas bila ada sesuatu yang ingin ditambahkan diluar formulir tugas yang diberikan. |
| user_id            | UUID      |                                                      | <b>Composite 1*</b> | Attribute penghubung dengan table user                                                                 |

Nama Tabel: **manager\_reviews**

| Nama Atribut       | Tipe Data | Panjang | Index     | Keterangan                                                            |
|--------------------|-----------|---------|-----------|-----------------------------------------------------------------------|
| id                 | UUID      | -       | <b>PK</b> | Identitas unik untuk setiap data review manajer.                      |
| submission_task_id | UUID      | -       |           | Foreign Key (FK). Referensi ke submission_tasks yang sedang direview. |

| Nama Atribut | Tipe Data | Panjang                              | Index         | Keterangan                                                             |
|--------------|-----------|--------------------------------------|---------------|------------------------------------------------------------------------|
| file_path    | String    | 255                                  |               | Lokasi penyimpanan file dokumen pendukung review (jika ada).           |
| notes        | Text      | -                                    |               | Catatan, umpan balik, atau hasil evaluasi dari manajer.                |
| review_by    | UUID      | -                                    | <b>B-Tree</b> | Foreign Key (FK). Referensi ke users (manajer yang memberikan review). |
| status       | Enum      | (approved, need_revision, rejected). |               | Status final keputusan manajer:                                        |
| review_at    | Datetime  |                                      |               | Waktu kapan review dilakukan.                                          |

---

Nama Tabel: **problem\_reports**

| Nama Atribut | Tipe Data | Panjang | Index         | Keterangan                                                     |
|--------------|-----------|---------|---------------|----------------------------------------------------------------|
| id           | UUID      | -       | <b>PK</b>     | Identitas unik untuk setiap laporan masalah.                   |
| name         | String    | 255     | <b>B-Tree</b> | Judul atau nama ringkas dari masalah yang dilaporkan.          |
| user_id      | UUID      | -       |               | Foreign Key (FK). Referensi ke users yang mengirimkan laporan. |
| description  | Text      | -       |               | Deskripsi lengkap atau detail kronologi                        |

| Nama Atribut | Tipe Data | Panjang                                        | Index         | Keterangan                                                          |
|--------------|-----------|------------------------------------------------|---------------|---------------------------------------------------------------------|
|              |           |                                                |               | masalah.                                                            |
| date_report  | Date      | -                                              |               | Tanggal ketika laporan masalah dibuat.                              |
| status       | Enum      | (open, in_progress, resolved, closed_invalid). |               | Status penanganan laporan:                                          |
| file_path    | String    | URL                                            |               | Alamat tautan atau lokasi penyimpanan dokumen/gambar bukti masalah. |
| priority     | Enum      | (low, medium, high).                           |               | Tingkat kepentingan laporan:                                        |
| answer       | Text      |                                                |               | Balasan laporan kendala oleh manajerial.                            |
| answer_by    | UUID      |                                                | <b>B-Tree</b> | Id user yang membalas laporan kendala.                              |
| answer_at    | Datetime  |                                                |               | Waktu kapan user menjawab laporan.                                  |

Nama Tabel: **sessions**

| Nama Atribut | Tipe Data | Panjang | Index         | Keterangan                                                     |
|--------------|-----------|---------|---------------|----------------------------------------------------------------|
| id           | UUID      | -       | <b>PK</b>     | Identitas unik untuk table session.                            |
| user_id      | UUID      | -       |               | Foreign Key (FK). Referensi ke users yang mengirimkan laporan. |
| token        | Text      |         | <b>B-Tree</b> | Data token yang                                                |

| Nama Atribut | Tipe Data | Panjang | Index | Keterangan                                                     |
|--------------|-----------|---------|-------|----------------------------------------------------------------|
|              |           |         |       | berhasil dibuat sebagai validasi auth                          |
| is_active    | Boolean   |         |       | Status apakah data session ini masih bisa digunakan atau tidak |
| expired_at   | Timestamp |         |       | Waktu kadaluarsa token.                                        |

#### 2.7.4 Referensi Status Sistem (Status Legend)

Bagian ini mendefinisikan arti fungsional dari setiap nilai *Enum* yang digunakan dalam sistem GENBA-HYOKA untuk menjamin konsistensi logika antar modul.

##### A. Status Pengiriman Tugas (submission\_tasks)

| Status              | Definisi Fungsional                     | Dampak Operasional                                                                 |
|---------------------|-----------------------------------------|------------------------------------------------------------------------------------|
| <b>draft</b>        | Laporan sedang dikerjakan oleh pegawai. | Data hanya terlihat oleh pegawai; belum dapat diulas manajer.                      |
| <b>submitted</b>    | Laporan telah dikirim secara formal.    | Data terkunci ( <i>read-only</i> ) bagi pegawai; muncul di antrian ulasan manajer. |
| <b>need_reviewd</b> | Manajer meminta perbaikan data.         | Pegawai diizinkan membuat versi baru (V2, V3, dst) berdasarkan feedback.           |
| <b>approved</b>     | Laporan diterima dan dinyatakan valid.  | Siklus tugas berakhir; data masuk ke perhitungan skor evaluasi.                    |
| <b>rejected</b>     | Laporan ditolak sepenuhnya.             | Tugas dianggap gagal; tidak masuk dalam akumulasi nilai positif.                   |

## B. Status Laporan Kendala (`problem_reports`)

| Status                | Definisi Fungsional              | Tindakan Selanjutnya                                                  |
|-----------------------|----------------------------------|-----------------------------------------------------------------------|
| <b>open</b>           | Masalah baru saja dilaporkan.    | Menunggu verifikasi atau penugasan dari pihak manajerial.             |
| <b>in_progress</b>    | Masalah sedang dalam penanganan. | Investigasi atau perbaikan fisik sedang dilakukan di lapangan.        |
| <b>resolved</b>       | Masalah telah selesai ditangani. | Tiket ditutup; solusi telah diterapkan dan diverifikasi.              |
| <b>closed_invalid</b> | Laporan dianggap tidak valid.    | Tiket ditutup tanpa tindakan (misal: duplikat atau informasi keliru). |

## C. Prioritas Laporan Masalah (`problem_reports.priority`)

Atribut ini digunakan untuk melakukan triaging (pemilahan) laporan kendala dari lapangan berdasarkan dampaknya terhadap integritas data dan kelangsungan tugas.

| Prioritas     | Definisi Dampak                                                                                                                                                        | Contoh Kasus Lapangan                                                                                                                          |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>low</b>    | <b>Kendala Prosedural/Administrasi.</b><br>Tugas utama tetap selesai tepat waktu, namun ada kondisi aturan atau SOP yang tidak terpenuhi secara sempurna.              | Terjadi kemacetan lalu lintas yang tidak terduga sehingga pengambilan tugas terlambat dari jadwal, namun inspeksi tetap berhasil dilakukan.    |
| <b>medium</b> | <b>Kendala Operasional Parsial.</b> Sebagian komponen tugas tidak dapat diselesaikan atau ada data pendukung yang hilang, sehingga hasil tugas menjadi tidak sempurna. | Client meminta penundaan pembayaran atau penolakan tindakan tertentu, sehingga bukti foto tindakan tidak bisa didapatkan sesuai skema laporan. |
| <b>high</b>   | <b>Kendala Fatal/Total.</b><br>Munculnya faktor eksternal berat yang membuat seluruh aktivitas tugas terhenti                                                          | Terjadi kecelakaan pada pengguna saat menuju lokasi atau kerusakan alat utama secara total, sehingga                                           |

| Prioritas | Definisi Dampak                       | Contoh Kasus Lapangan              |
|-----------|---------------------------------------|------------------------------------|
|           | seketika dan tidak dapat dilanjutkan. | inspeksi lapangan batal dilakukan. |

#### D. Tipe Notifikasi (`notifications.type`)

Digunakan oleh aplikasi untuk menentukan gaya visual (seperti warna banner atau suara peringatan) yang muncul di perangkat pengguna.

| Tipe          | Deskripsi Fungsi                             | Contoh Kejadian                                               |
|---------------|----------------------------------------------|---------------------------------------------------------------|
| <b>Info</b>   | Berita umum yang bersifat pasif.             | Pengumuman kebijakan baru atau tips harian.                   |
| <b>Normal</b> | Informasi terkait alur kerja rutin.          | Adanya penugasan tugas baru ( <code>task_assignment</code> ). |
| <b>Urgent</b> | Peringatan yang membutuhkan tindakan segera. | Laporan yang butuh revisi atau adanya laporan masalah baru.   |

#### E. Keputusan Review Manajer (`manager_reviews.status`)

Status ini berfungsi sebagai pemicu otomatis (*trigger*) yang mengubah kondisi dokumen laporan pada tabel `submission_tasks`.

| Status Review        | Makna Keputusan Manajer                             | Dampak pada <code>submission_tasks</code>                  |
|----------------------|-----------------------------------------------------|------------------------------------------------------------|
| <b>approved</b>      | Laporan dinyatakan valid dan memenuhi standar.      | Status laporan berubah menjadi <code>approved</code> .     |
| <b>need_revision</b> | Terdapat data yang keliru atau kurang lengkap.      | Status laporan berubah menjadi <code>need_reviewd</code> . |
| <b>rejected</b>      | Laporan dianggap tidak valid atau menyalahi aturan. | Status laporan berubah menjadi <code>rejected</code> .     |

## 2.8 Spesifikasi Teknologi

Sistem menggunakan arsitektur dan pustaka berikut untuk memastikan performa yang optimal:

- **Runtime:** Bun
- **Framework:** Elysia.js
- **ORM:** Drizzle ORM

- **Database:** PostgreSQL (v15+)
- **Pustaka Keamanan & Validasi:** Jose (JWT), TypeBox/Zod (Validasi), Bun.password (Hashing).
- **Pdf Engine :** Puppeteer 24 +.

## 2.9 Strategi Penanganan Kesalahan

Penanganan kesalahan dikelola secara terpusat (*Global Error Handling*) melalui *middleware*. Sistem mengembalikan kode HTTP standar dengan format pesan yang seragam tanpa mengekspos informasi sensitif dari *server* kepada klien.

Berikut adalah tabel referensi kode status HTTP yang digunakan dalam sistem:

| Kode HTTP | Nama Status  | Deskripsi Kondisi                                                                  | Contoh Respons                                                         |
|-----------|--------------|------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| 200       | OK           | Permintaan berhasil diproses oleh server tanpa masalah.                            | { "success": true, "message": "Data berhasil diambil" }                |
| 201       | Created      | Permintaan berhasil dan sebuah data/sumber daya baru telah terbuat.                | { "success": true, "message": "Tugas berhasil dibuat" }                |
| 400       | Bad Request  | Terdapat kesalahan logika atau format pada data input dari klien.                  | { "success": false, "message": "Input tidak valid" }                   |
| 401       | Unauthorized | Klien tidak terautentikasi (Token JWT tidak disertakan atau sudah kedaluwarsa).    | { "success": false, "message": "Sesi tidak ditemukan, silakan login" } |
| 403       | Forbidden    | Klien terautentikasi tetapi tidak memiliki hak akses yang sesuai (Kegagalan RBAC). | { "success": false, "message": "Anda tidak memiliki izin" }            |
| 404       | Not Found    | Data atau <i>endpoint</i> yang diminta tidak dapat ditemukan di dalam server.      | { "success": false, "message": "Data tidak ditemukan" }                |

| Kode HTTP | Nama Status           | Deskripsi Kondisi                                                                   | Contoh Respons                                                                    |
|-----------|-----------------------|-------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| 422       | Unprocessable Entity  | Permintaan ditolak karena gagal melewati validasi skema data (Zod/TypeBox).         | <code>{ "success": false, "message": "Format data laporan salah" }</code>         |
| 500       | Internal Server Error | Terjadi kegagalan atau kerusakan pada internal logika server yang tidak tertangani. | <code>{ "success": false, "message": "Terjadi kesalahan internal server" }</code> |

## 2.10 Strategi Generasi dan Penyimpanan Dokumen

**Alur Kerja Generasi PDF (Server-Side)** Sistem menggunakan pendekatan **Headless Chrome Rendering**. Alur eksekusi adalah sebagai berikut:

1. **Data Fetching:** Backend mengambil data dari tabel `submissions`, `users`, dan `tasks`.
2. **HTML Templating:** Data dimasukkan ke dalam *template* HTML yang sudah didesain sesuai standar laporan perusahaan.
3. **Rendering:** Puppeteer menjalankan instruksi untuk mencetak (*print-to-pdf*) halaman HTML tersebut secara asinkron.
4. **Storage:** File PDF disimpan ke direktori penyimpanan, dan jalurnya (*file path*) dicatat ke kolom `evaluation_reports.file_path`.

## BAB 3: Antarmuka dan Integrasi Sistem

Bab ini mendefinisikan standar komunikasi data antara klien dan *server* untuk menjamin konsistensi, keamanan, dan kecepatan transmisi informasi.

### 3.1 Arsitektur REST API

Sistem mengadopsi prinsip *Representational State Transfer* (REST) yang menekankan pada skalabilitas, kemudahan integrasi, dan kemandirian antara klien dan server.

#### 3.1.1 Resource-Based Routing (Nouns, not Verbs)

API dirancang berdasarkan "Sumber Daya" (*Resources*), bukan berdasarkan tindakan. Nama *endpoint* harus menggunakan kata benda jamak (*plural nouns*) untuk menjaga konsistensi.

- **Benar:** `GET /v1/tasks` (Mengambil daftar tugas)
- **Salah:** `GET /v1/get-all-tasks` (Menggunakan kata kerja)

#### 3.1.2 Pemetaan HTTP Methods ke CRUD

Setiap operasi pada data dipetakan secara ketat menggunakan metode HTTP yang sesuai:

- **GET:** Digunakan untuk membaca atau mengambil data. Bersifat *idempotent* (tidak mengubah data di server).
- **POST:** Digunakan untuk membuat sumber daya baru (misal: mengirimkan `submission_tasks`).
- **PUT / PATCH:** Digunakan untuk memperbarui data yang sudah ada.
- **DELETE:** Digunakan untuk menghapus data (dalam sistem ini, menjalankan fungsi *Soft Delete*).

#### 3.1.3 Versi API (Versioning)

Penggunaan `/v1/` pada jalur URL bersifat wajib. Hal ini menjamin bahwa jika di masa depan terdapat perubahan besar (*breaking changes*) pada struktur data, versi lama tetap bisa berjalan dengan jalur `/v1/` sementara versi baru menggunakan jalur `/v2/`.

#### 3.1.4 Statelessness (Kemandirian Sesi)

Backend GENBA-HYOKA beroperasi secara **Stateless**, di mana setiap permintaan (*request*) dari klien ke server bersifat mandiri dan harus mengandung seluruh informasi yang diperlukan untuk diproses. Server tidak menyimpan *session state* atau konteks pengguna di dalam memori internal (RAM), sehingga memungkinkan skalabilitas sistem yang lebih tinggi dan distribusi beban yang lebih merata.

Proses otentikasi dan otorisasi dilakukan sepenuhnya melalui token JWT yang wajib disertakan pada setiap *Header* permintaan:

- **Format Header:** `Authorization: Bearer <token_jwt>`

Untuk mendukung kemandirian ini tanpa mengorbankan performa, *payload* JWT dirancang

untuk membawa "**Hak Akses Efektif**" yang sudah dikalkulasi di awal (*pre-calculated*). Dengan menyematkan hak akses langsung di dalam token, server dapat melakukan verifikasi izin akses secara instan tanpa perlu melakukan kueri ulang ke basis data.

Struktur data otorisasi menggunakan objek *key-value*, di mana setiap *key* merepresentasikan kode modul (3 karakter) dan *value* berisi angka integer hasil penggabungan bitmask.

#### Contoh Payload Efektif:

```
{
  "sub": "user-uuid-123",
  "name": "Budi Santoso",
  "prm": {
    "rpt": 3,    // Reports: Read + Create
    "tsk": 5,    // Tasks: Read + Update
    "usr": 1,    // Users: Read Only
    "cfg": 0     // Config: No Access
  },
  "role": {
    { code: 'sup', name: 'Super Admin', type: 'super_admin' },
    { code: 'adm', name: 'Administrator', type: 'admin' },
    { code: 'man', name: 'Manager', type: 'manager' },
    { code: 'emp', name: 'Employee', type: 'employee' },
  },
  "iat": 1713237500,
  "exp": 1713241100
}
```

Kamus Pemetaan Hak Akses Berdasarkan Tipe Role

#### Super Admin

| Nama table       | Kode table | Hak Akses  |
|------------------|------------|------------|
| User             | USR        | C, R, U, D |
| Role             | ROL        | C, R, U, D |
| Company Profile  | CPY        | C, R, U, D |
| Notification     | NFT        | R, U       |
| Audit log        | AUD        | R          |
| Task Definition  | TDF        | C, R, U, D |
| Task Assignments | TAS        | C, R, U, D |
| Submission Task  | SUB        | C, R, U, D |
| Problem Report   | PRB        | C, R, U, D |
| Manager Reviews  | REV        | C, R, U, D |
| Template Report  | TMP        | C, R, U, D |

#### Admin

| Nama table      | Kode table | Hak Akses  |
|-----------------|------------|------------|
| User            | USR        | C, R, U, D |
| Role            | ROL        | C, R, U, D |
| Company Profile | CPY        | C, R, U, D |
| Notification    | NFT        | R, U       |
| Audit log       | AUD        | R          |

#### Manager

| Nama table       | Kode table | Hak Akses  |
|------------------|------------|------------|
| User             | USR        | R          |
| Role             | ROL        | R          |
| Company Profile  | CPY        | R          |
| Notification     | NFT        | R, U       |
| Audit log        | AUD        | R          |
| Task Definition  | TDF        | C, R, U, D |
| Task Assignments | TAS        | C, R, U, D |
| Submission Task  | SUB        | R          |
| Problem Report   | PRB        | R, U       |
| Manager Reviews  | REV        | C, R, U, D |
| Template Report  | TMP        | C, R, U, D |

#### Employee

| Nama table       | Kode table | Hak Akses  |
|------------------|------------|------------|
| User             | USR        | R          |
| Role             | ROL        | R          |
| Company Profile  | CPY        | R          |
| Notification     | NFT        | R, U       |
| Audit log        | AUD        | R          |
| Task Definition  | TDF        | R          |
| Task Assignments | TAS        | R,         |
| Submission Task  | SUB        | C, R, U, D |
| Problem Report   | PRB        | C, R, U, D |
| Manager Reviews  | REV        | R          |

### 3.1.5 Keamanan Transmisi (HTTPS)

Seluruh titik akses API wajib diakses melalui protokol HTTPS. Hal ini memastikan bahwa data sensitif seperti *password* dan laporan lapangan dienkripsi selama perjalanan dari perangkat pengguna ke server, guna mencegah serangan *Man-in-the-Middle*.

### 3.1.6 Strategi Paginasi (Pagination Strategy)

Sistem mengimplementasikan dua metode paginasi untuk memastikan efisiensi penarikan data berdasarkan karakteristik entitasnya:

#### A. Offset-Based Pagination (Standard)

Digunakan untuk data yang bersifat relatif statis atau memerlukan navigasi halaman spesifik (seperti daftar tugas atau laporan di panel admin).

- **Parameter:** page (nomor halaman) dan limit (jumlah data per halaman).
- **Contoh Endpoint:** GET /v1/submissions?page=1&limit=20
- **Kelebihan:** Memungkinkan pengguna untuk melompat ke halaman tertentu.

## Standar Metadata Respon

Untuk menjaga konsistensi di sisi klien (*Frontend*), setiap permintaan data dalam jumlah besar akan selalu menyertakan objek meta di dalam *payload* sukses:

JSON

```
{
  "success": true,
  "message": "Data retrieved successfully",
  "data": [...],
  "meta": {
    "total": 150,           // Total seluruh data (untuk Offset)
    "current_page": 1,     // Halaman saat ini (untuk Offset)
    "last_page": 8,        // Halaman terakhir (untuk Offset)
    "limit": 20            // limit data yang diambil
  }
}
```

## B. Cursor-Based Pagination (Performance)

Digunakan untuk data dengan volume tinggi yang terus bertambah secara cepat, seperti daftar notifikasi atau log aktivitas lapangan. Hal ini dilakukan untuk menghindari masalah "data duplikat" saat pengguna melakukan *scrolling* sementara data baru masuk.

- **Parameter:** cursor (menggunakan ID atau *timestamp* terakhir yang diterima) dan limit.
- **Contoh Endpoint:** GET /v1/notifications?cursor=uuid-123&limit=20
- **Kelebihan:** Performa kueri database yang konstan meskipun data mencapai jutaan baris.

## Standar Metadata Respon

Untuk menjaga konsistensi di sisi klien (*Frontend*), setiap permintaan data dalam jumlah besar akan selalu menyertakan objek meta di dalam *payload* sukses:

JSON

```
{
  "success": true,
  "message": "Data retrieved successfully",
  "data": [...],
  "meta": {
    "next_cursor": "uuid-123",
    "has_more": true,
    "limit": 20
  }
}
```

## 3.2 Protokol Koneksi dan Integrasi WebSocket

Bagian ini mengatur bagaimana aplikasi klien (Mobile/Web) melakukan jabat tangan (*handshake*) dan mengelola sesi komunikasi *real-time*.

**3.2.1 Autentikasi: One-Time Ticket (OTT)** Demi keamanan, JWT tidak dikirimkan langsung melalui *query parameter* WebSocket. Sistem menggunakan mekanisme tiket sekali pakai:

1. **Request Ticket:** Klien melakukan POST `/v1/auth/ws-ticket` dengan *Header Authorization* (JWT).
2. **Ticket Issued:** Server mengembalikan `ticket_id` yang berlaku selama 60 detik.
3. **Connection:** Klien membuka koneksi ke `wss://api.genba.com/ws?ticket={ticket_id}`.

**3.2.2 Struktur Payload Pesan** Semua pesan yang dikirim melalui WebSocket menggunakan format JSON seragam untuk memudahkan *parsing* di sisi klien:

JSON

```
{
  "event": "string",          // Contoh: "NEW_TASK", "STATUS_UPDATED"
  "data": "object",          // Data terkait event
  "timestamp": "iso8601"     // Waktu pengiriman dari server
}
```

**3.2.3 Protokol Reconnection (Client-Side)** Aplikasi klien wajib mengimplementasikan strategi **Exponential Backoff** jika terjadi kegagalan koneksi:

- Percobaan ulang pertama dilakukan setelah 1 detik.
- Jeda meningkat secara eksponensial (2s, 4s, 8s) hingga batas maksimal 30 detik.
- Setiap kali status berubah menjadi `CONNECTED`, aplikasi harus memicu fungsi *Sync* untuk memastikan tidak ada informasi yang tertinggal.

## 3.3 Standar Format Respon

Semua respons API menggunakan struktur JSON yang konsisten untuk memudahkan integrasi pada sisi klien.

- **Sukses:**

```
{
  "success": true,
  "message": "...",
}
```

```

    "data": { ... }
  }

  • Gagal:

  {
    "success": false,
    "message": "...",
    "error_code": "...",
    "details": [ ... ]
  }

```

### 3.4 Logika Pendefinisian Tugas

Sistem Genba-Hyoka dirancang dengan mengedepankan fleksibilitas operasional. Oleh karena itu, sistem ini dilengkapi dengan fitur **Kustom Formulir (Dynamic Form Builder)**. Melalui fitur ini, pengguna dengan peran **manajerial** memiliki wewenang penuh untuk merancang, membuat, dan menyesuaikan templat tugas atau formulir pelaporan sesuai dengan Standar Operasional Prosedur (SOP) tanpa memerlukan intervensi dari tim pengembang (*no-code approach*).

Seluruh logika pendefinisian tugas disimpan di dalam tabel `task_definition` menggunakan dua atribut utama bertipe **JSONB**, yaitu `form_schema` dan `procedures`.

#### 3.4.1 Atribut `form_schema` (Struktur Formulir Dinamis)

Atribut `form_schema` mendefinisikan struktur antarmuka dan validasi dari formulir yang akan dirender pada aplikasi sisi klien. Elemen-elemen formulir dibungkus di dalam *array fields*.

##### A. Atribut Dasar dan Kendali Elemen

Setiap objek di dalam *fields* wajib memiliki identitas dan dapat dikendalikan aksesnya:

- **id**: Identifier unik (berformat *snake\_case*) yang bertindak sebagai kunci (*key*) data saat pelaporan disimpan.
- **label**: Nama kolom yang dirender pada antarmuka.
- **type**: Jenis komponen antarmuka (misal: *text*, *number*, *camera*).
- **description** (*Optional*): Teks instruksi panduan bagi petugas lapangan.
- **is\_locked** (*Optional*): Parameter *Boolean*. Jika *true*, elemen bersifat hanya-baca (*read-only*).

- **default\_value** (*Optional*): Nilai bawaan elemen. Mendukung penggunaan *placeholder* sistem (contoh: {{SYSTEM\_DATE}}, {{USER\_NAME}}) yang diterjemahkan saat formulir dimuat.

## B. Spesifikasi Tipe Input dan Validasi (rules)

Logika validasi ditetapkan di dalam objek rules. Parameter required (Boolean) wajib ada pada setiap elemen. Validasi lanjutan bergantung pada nilai type:

| Tipe Input  | Fungsi                         | Validasi Khusus (rules)                       |
|-------------|--------------------------------|-----------------------------------------------|
| text        | String tunggal (nama, kode).   | min_length, max_length, regex_pattern.        |
| textarea    | Teks panjang (catatan).        | min_length, max_length.                       |
| number      | Nilai numerik.                 | min, max, allow_decimal.                      |
| datetime    | Waktu dan tanggal.             | disable_future_dates, min_date, max_date.     |
| geolocation | Pencatatan koordinat GPS.      | fetch_method (auto atau manual).              |
| camera      | Foto langsung ( <i>live</i> ). | allow_gallery (Boolean), max_size_mb.         |
| file        | Unggahan dokumen (PDF).        | max_size_mb, allowed_extensions.              |
| signature   | Tanda tangan digital.          | ( <i>Hanya parameter required</i> )           |
| dropdown    | Pilihan tunggal.               | ( <i>Diatur via konfigurasi data_source</i> ) |
| checkbox    | Pilihan ganda.                 | min_selections, max_selections.               |

## C. Logika Visibilitas Bersyarat (show\_if)

Elemen dapat diatur agar hanya muncul jika kondisi elemen lain terpenuhi, untuk menjaga formulir tetap ringkas.

- field: ID elemen acuan.
- operator: Logika komparasi (==, !=, >, <, contains).
- value: Nilai yang memicu elemen ditampilkan.

## D. Sumber Data dan Relasi Terpadu (data\_source & depends\_on)

Khusus untuk elemen bertipe pilihan (dropdown, checkbox), opsi diatur melalui objek data\_source yang mendukung relasi data bertingkat (*Cascading*) dengan logika terpadu.

### 1. Sumber Data (type):

- **static**: Opsi dikonfigurasi langsung di dalam JSON melalui *array options*.
- **dynamic**: Opsi ditarik dari API *backend* melalui endpoint, lengkap dengan atribut pemetaan label\_key dan value\_key.

### 2. Relasi Terpadu (depends\_on):

Berisi *array* objek (memungkinkan relasi multi-kondisi) yang memuat field (ID induk acuan) dan param\_name (Kunci Relasi).

- **Pada Data Dinamis:** param\_name disisipkan sebagai parameter kueri URL ke *backend* (Contoh: ?param\_name=nilai\_induk).
- **Pada Data Statis:** param\_name bertindak sebagai kunci filter (*Foreign Key* logis) pada setiap objek di dalam *array* options. Opsi hanya ditampilkan jika nilai properti yang bernama sama dengan param\_name cocok dengan jawaban pada elemen induk.

Contoh JSON:

```
{
  "fields": [
    {
      "id": "nama_inspektur",
      "label": "Nama Inspektur Lapangan",
      "type": "text",
      "is_locked": true,
      "default_value": "{{USER_NAME}}",
      "rules": { "required": true }
    },
    {
      "id": "cabang_inspeksi",
      "label": "Lokasi Cabang",
      "type": "dropdown",
      "data_source": {
        "type": "dynamic",
        "endpoint": "/v1/branches",
        "label_key": "nama_cabang",
        "value_key": "id"
      },
      "rules": { "required": true }
    },
    {
      "id": "mesin_target",
      "label": "Pilih Mesin",
      "type": "dropdown",
      "data_source": {
        "type": "dynamic",
        "endpoint": "/v1/assets",
        "label_key": "nama_mesin",
        "value_key": "id",
        "depends_on": {
          "field": "cabang_inspeksi",
          "param_name": "branch_id"
        }
      },
      "rules": { "required": true }
    },
    {
      "id": "status_kondisi",
      "label": "Kondisi Aktual",
      "type": "dropdown",
      "data_source": {
```

```

        "type": "static",
        "options": [
            { "label": "Berfungsi Optimal", "value": "optimal" },
            { "label": "Perlu Perbaikan", "value": "rusak" }
        ]
    },
    "rules": { "required": true }
},
{
    "id": "foto_kerusakan",
    "label": "Ambil Foto Kerusakan",
    "type": "camera",
    "show_if": {
        "field": "status_kondisi",
        "operator": "==",
        "value": "rusak"
    },
    "rules": {
        "required": true,
        "allow_gallery": false,
        "max_size_mb": 5
    }
},
{
    "id": "tanda_tangan_validasi",
    "label": "Validasi Manajer Area",
    "type": "signature",
    "rules": { "required": true }
},
{
    "id": "tipe_masalah",
    "label": "Tipe Masalah",
    "type": "dropdown",
    "data_source": {
        "type": "static",
        "options": [
            { "label": "Perangkat Keras", "value": "hw" },
            { "label": "Perangkat Lunak", "value": "sw" }
        ]
    },
    "rules": { "required": true }
},
{
    "id": "detail_masalah",
    "label": "Detail Masalah",
    "type": "dropdown",
    "data_source": {
        "type": "static",
        "depends_on": [
            { "field": "tipe_masalah", "param_name": "kategori" }
        ],
        "options": [
            { "label": "Layar Retak", "value": "retak", "kategori": "hw" },
            { "label": "Baterai Kembung", "value": "baterai", "kategori": "hw" },
            { "label": "Aplikasi Crash", "value": "crash", "kategori": "sw" },
            { "label": "Gagal Login", "value": "login", "kategori": "sw" }
        ]
    }
}

```

```

    ],
    },
    "rules": { "required": true }
  }
]
}

```

### 3.4.2 Atribut procedures (Logika Bisnis & Penjadwalan)

Atribut procedures adalah mesin aturan (*Rules Engine*) berbasis JSONB yang mengatur perilaku tugas di luar struktur formulir. Aturan ini menentukan kapan tugas tersedia, siapa yang memverifikasi, dan aturan terkait pengisian formulir.

Konfigurasi ini dibagi menjadi tiga pilar utama: **Generation**, **SLA Policy**, dan **Workflow**.

| Pilar                        | Atribut        | Tipe Data   | Opsi / Deskripsi                                                                                                                         |
|------------------------------|----------------|-------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Generation</b>            | strategy       | String      | jit_backend (Just-In-Time): Tugas dibuat saat petugas mengakses aplikasi. (saat ini baru ada satu opsi)                                  |
|                              | active_days    | Array [Int] | Hari aktif (1=Senin, 7=Minggu). Contoh: [1,2,3,4,5].                                                                                     |
| <b>submissi<br/>on_rules</b> | start_at       | String      | Format HH:mm. Jam paling awal tugas boleh dikerjakan.                                                                                    |
|                              | duration_hours | Integer     | Masa berlaku tugas dalam jam.                                                                                                            |
|                              | start_time_ref | String      | on_generated: Hitung mundur sejak start_at.<br>on_started: Hitung mundur sejak petugas klik "Mulai".<br>none: jika tidak ada batas waktu |
|                              | instructions   | String      | Deskripsi tambahan untuk membantu proses pengisian formulir                                                                              |
|                              | support_file   | String      | Url yang berisi dokumen pendukung untuk melakukan pengisian formulir                                                                     |

| Pilar           | Atribut         | Tipe Data   | Opsi / Deskripsi                                                |
|-----------------|-----------------|-------------|-----------------------------------------------------------------|
|                 | multi_entry     | Boolean     | Informasi apakah formulir bisa di isi berkali kali atau tidak.  |
| <b>Workflow</b> | requires_review | Boolean     | Jika true, membutuhkan validasi manajer setelah <i>submit</i> . |
|                 | approval_role   | Array [Str] | Role ID yang berhak memvalidasi berisi UUID                     |

Contoh JSON

```
{
  "generation": {
    "strategy": "jit_backend",
    "active_days": [1, 2, 3, 4, 5]
  },
  "submission_rules": {
    "start_at": "08:00",
    "duration_hours": 4,
    "start_time_ref": "on_started",
    "instructions": ".....",
    "support_file": "url_file pendukung",
    "multi_entry": true,
  },
  "workflow": {
    "requires_review": true,
    "approval_role": ["man", "sup"]
  }
}
```

### Alur Perpindahan Status (State Machine)

Alur ini melibatkan tabel `task_assignment` (sebagai pengatur status utama) dan `task_submission` (sebagai penampung data).

1. **DRAFT**: Tugas baru saja digenerasi oleh sistem (Lazy Generation).
2. Petugas sudah mulai membuka dan mengisi formulir di aplikasi
  - *Sistsem melakukan proses simpan dan kirim*
  - *Sistem melakukan pengecekan `requires_review: true` atau tidak.*
3. Jika true maka status nya **NEED REVIEW**
4. Jika false maka status **SUBMITTED**

#### 5. VERIFIKASI MANAJER (Decision Point):

- **APPROVE:** tugas selesai.
- **REJECT / REVISE:** Status berubah menjadi **REVISION REQUIRED**. Tugas dikembalikan ke petugas untuk diperbaiki.

### 3.5 Dokumentasi API

Dokumentasi API dihasilkan secara otomatis menggunakan Swagger/Scalar yang terintegrasi dengan Elysia.js, sehingga spesifikasi teknis selalu mutakhir untuk kebutuhan tim pengembang.

